

LAPORAN PRATIKUM

STRUKTUR DATA

“Pratikum Pekan 4”

Disusun Oleh:

Rafikhul Ramadhan

2511533012

Dosen Pengampu : Dr. Wahyudi, S.T, M.T.

Asisten Praktikum : Muhammad Galis Avero



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2025

KATA PENGANTAR

Segala puji dan syukur saya panjatkan ke hadirat Allah SWT atas limpahan rahmat, taufik, dan karunia-Nya sehingga penulis dapat menyelesaikan laporan praktikum mata kuliah Struktur Data ini dengan baik dan tepat waktu.

Laporan praktikum ini disusun sebagai salah satu bentuk pemenuhan tugas akademik, sekaligus sebagai sarana untuk memperdalam pemahaman penulis terhadap konsep-konsep dasar dalam struktur data. Melalui praktikum ini, saya mempelajari berbagai materi penting seperti penggunaan array, stack, queue, linked list, serta bagaimana mengimplementasikannya dalam bahasa pemrograman. Selain itu, kegiatan praktikum juga melatih kemampuan berpikir logis, sistematis, dan terstruktur dalam menyelesaikan permasalahan.

Saya menyadari bahwa dalam penyusunan laporan ini masih terdapat berbagai kekurangan, baik dari segi penulisan maupun isi. Oleh karena itu, saya sangat mengharapkan kritik dan saran yang membangun guna perbaikan di masa yang akan datang.

Pada kesempatan ini, saya juga ingin menyampaikan rasa terima kasih kepada dosen pengampu, asisten praktikum, serta teman-teman yang telah memberikan bimbingan, dukungan, dan bantuan selama proses praktikum hingga penyusunan laporan ini.

Akhir kata, semoga laporan praktikum ini dapat memberikan manfaat bagi penulis khususnya, serta bagi para pembaca pada umumnya. Penulis berharap laporan ini juga dapat digunakan sebagai referensi dalam memahami materi praktikum, baik pada mata kuliah Struktur Data maupun praktikum lainnya.

Padang, 29 April 2026

Rafikhul Ramadhan

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI	ii
BAB I PENDAHULUAN	1
1.1 Latar Belakang.....	1
1.2 Tujuan	1
1.3 Manfaat	1
BAB II PEMBAHASAN.....	2
2.1 Langkah Kerja.....	2
2.2 QueueArray_2511533012	2
2.3 QueueArrayDriver_2511533012	4
2.4 QueueLinkedList_2511533012.....	5
2.5 ReverseData_2511533012.....	6
2.6 IterasiQueue_2511533012	7
BAB III KESIMPULAN	9
3.1 Ringkasan.....	9
3.2 Kesimpulan.....	9
DAFTAR PUSTAKA	10

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pada praktikum pekan ke-4, mahasiswa mempelajari konsep struktur data **Queue (antrian)** dalam bahasa pemrograman Java. Queue merupakan struktur data yang bekerja dengan prinsip **FIFO (First In First Out)**, yaitu data yang pertama masuk akan menjadi data pertama yang keluar. Struktur data ini banyak digunakan dalam berbagai sistem komputer seperti pengelolaan antrian proses, sistem pelayanan, dan pengolahan data secara berurutan.

Melalui praktikum ini, mahasiswa mempelajari implementasi queue menggunakan array maupun LinkedList, serta memahami operasi dasar seperti menambah data, menghapus data, melihat elemen terdepan, dan melakukan iterasi data.

1.2 Tujuan

Tujuan dari praktikum pekan ke-4 adalah untuk memahami konsep dasar struktur data queue, mengimplementasikan queue menggunakan array dan LinkedList, serta memahami operasi-operasi dasar pada queue dalam bahasa Java.

1.3 Manfaat

Manfaat dari praktikum ini adalah mahasiswa mampu memahami cara kerja struktur data queue, meningkatkan kemampuan dalam pengelolaan data secara terurut, serta mampu mengimplementasikan queue dalam penyelesaian masalah sederhana menggunakan Java.

BAB II

PEMBAHASAN

2.1 Langkah Kerja

Tahapan yang dilakukan dalam pratikum ini adalah sebagai berikut:

1. Menjalankan aplikasi IDE (seperti Eclipse atau IntelliJ IDEA) atau menggunakan text editor yang mendukung bahasa Java.
2. Membuat sebuah project baru dengan nama **pekan4_2511533012**.
3. Menambahkan tiga buah file Java, yaitu QueueArray_2511533012.java, QueueArrayDriver_2511533012.java, QueueLinkedList_2511533012.java, IterasiQueue_2511533012.java, ReverseData_2511533012.java,
4. Menuliskan kode program sesuai dengan instruksi yang diberikan pada masing-masing file.
5. Melakukan proses kompilasi menggunakan *Java compiler* agar program dapat dijalankan.
6. Menjalankan program untuk mengamati hasil keluaran sesuai dengan logika yang dibuat.
7. Mengevaluasi hasil eksekusi pada *console* serta menarik kesimpulan dari setiap percobaan yang dilakukan.

2.2 QueueArray_2511533012

```
package pekan4_2511533012;

public class QueueArray_2511533012 {
    int front, rear, size;
    int capacity;
    int array[];

    public QueueArray_2511533012(int capacity) {
        this.capacity = capacity;
        front = this.size = 0;
        rear = capacity - 1;
        array = new int [this.capacity];
    }

    boolean isFull_3012(QueueArray_2511533012 queue) {
```

```

        return (queue.size == queue.capacity);
    }

    boolean isEmpty_3012(QueueArray_2511533012 queue) {
        return (queue.size == 0);
    }

    void enqueue_3012(int item) {
        if (isFull_3012(this))
            return;
        this.rear = (this.rear + 1) % this.capacity;
        this.array[this.rear] = item;
        this.size = this.size + 1;
        System.out.println(item + " enqueue to queue");
    }

    int dequeue_3012() {
        if (isEmpty_3012(this))
            return Integer.MIN_VALUE;
        int item = this.array[this.front];
        this.front = (this.front + 1) % this.capacity;
        this.size = this.size - 1;
        return item;
    }

    int front_3012() {
        if (isEmpty_3012(this))
            return Integer.MIN_VALUE;

        return this.array[this.front];
    }

    int rear_3012() {
        if (isEmpty_3012(this))
            return Integer.MIN_VALUE;

        return this.array[this.rear];
    }

    void display_3012() {
        int i;
        if (front == rear) {
            System.out.printf("\nAntrian Kosong\n");
            return;
        }

        for (i = front; i < rear; i++) {
            System.out.printf("%d <-- ", array[i]);
        }
    }
}

```

Analisis:

Program ini merupakan implementasi queue menggunakan **array**.

Class ini memiliki operasi dasar queue seperti enqueue untuk menambahkan data, dequeue untuk menghapus data, front untuk melihat elemen terdepan, rear untuk melihat elemen terakhir, dan display untuk menampilkan isi queue. Program juga menggunakan pengecekan kondisi queue penuh dan kosong.

Kesimpulan: Program ini menjelaskan implementasi queue secara manual menggunakan array.

2.3 QueueArrayDriver_2511533012

```
package pekan4_2511533012;

public class QueueArrayDriver_2511533012 {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        QueueArray_2511533012 queue_3012 = new
QueueArray_2511533012(1000);
        queue_3012.enqueue_3012(10);
        queue_3012.enqueue_3012(20);
        queue_3012.enqueue_3012(30);
        queue_3012.enqueue_3012(40);
        System.out.println("Item di depan " +
queue_3012.front_3012());
        System.out.println("Item paling belakang " +
queue_3012.rear_3012());
        System.out.println("tampilan queue");
        queue_3012.display_3012();
        System.out.println();
        System.out.println(queue_3012.dequeue_3012() + "
dihapus dari queue");
        System.out.println("Item di depan " +
queue_3012.front_3012());
        System.out.println("Item di belakang " +
queue_3012.rear_3012());
        System.out.println("tampilan queue setelah satu data
dihapus");
        queue_3012.display_3012();

    }

}
```

Output (2.2. QueueArray_2511533012.java) :

```
10 enqueue to queue
20 enqueue to queue
30 enqueue to queue
40 enqueue to queue
Item di depan 10
Item paling belakang 40
tampilan queue
10 <-- 20 <-- 30 <--
10 dihapus dari queue
Item di depan 20
Item di belakang 40
tampilan queue setelah satu data dihapus
20 <-- 30 <--
```

Analisis :

Program ini berfungsi sebagai **driver** untuk menguji class QueueArray. Program melakukan beberapa operasi queue seperti menambahkan data (enqueue), menghapus data (dequeue), melihat elemen depan dan belakang, serta menampilkan isi queue sebelum dan sesudah penghapusan data.

Kesimpulan: Program ini digunakan untuk menguji fungsi-fungsi queue berbasis array.

2.4 QueueLinkedList_2511533012

```
package pekan4_2511533012;

import java.util.LinkedList;
import java.util.Queue;

public class QueueLinkedList_2511533012 {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        Queue<Integer> q_3012 = new LinkedList<>();

        for (int i = 0; i < 6; i++)
            q_3012.add(i);

        System.out.println("Elemen Antrian " + q_3012);

        int hapus = q_3012.remove();
        System.out.println("Hapus elemen = " + hapus);
        System.out.println(q_3012);
    }
}
```

```

        int depan = q_3012.peek();
        System.out.println("Kepala Antrian = " + depan);

        int banyak = q_3012.size();
        System.out.println("Size Antrian = " + banyak);
    }
}

```

Output :

```

Elemen Antrian [0, 1, 2, 3, 4, 5]
Hapus elemen = 0
[1, 2, 3, 4, 5]
Kepala Antrian = 1
Size Antrian = 5

```

Analisis :

Program ini mengimplementasikan struktur data **Queue menggunakan LinkedList**.

Program menambahkan beberapa data ke dalam queue menggunakan method `add()`, kemudian menampilkan isi queue. Selanjutnya dilakukan operasi `remove()` untuk menghapus elemen terdepan dan `peek()` untuk melihat elemen paling depan tanpa menghapusnya. Program juga menampilkan jumlah data menggunakan method `size()`.

Kesimpulan: Program ini menunjukkan operasi dasar queue menggunakan `LinkedList`.

2.5 ReverseData_2511533012

```

package pekan4_2511533012;

import java.util.LinkedList;
import java.util.Queue;
import java.util.Stack;

public class ReverseData_2511533012 {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        Queue<Integer> q_3012 = new LinkedList<Integer>();
    }
}

```

```

        q_3012.add(1);
        q_3012.add(2);
        q_3012.add(3);
        System.out.println("Sebelum reverse" + q_3012);
        Stack<Integer> s_3012 = new Stack<Integer>();
        while (!q_3012.isEmpty()) {
            s_3012.push(q_3012.remove());
        }
        while (!s_3012.isEmpty()) {
            q_3012.add(s_3012.pop());
        }
        System.out.println("Sesudah reverse = " + q_3012);
    }
}

```

Output :

```

Sebelum reverse[1, 2, 3]
Sesudah reverse = [3, 2, 1]

```

Analisis :

Program ini bertujuan untuk membalik urutan data pada queue menggunakan bantuan struktur data **Stack**.

Data queue dipindahkan terlebih dahulu ke stack menggunakan operasi push, kemudian dikembalikan lagi ke queue menggunakan pop. Karena stack bekerja dengan prinsip LIFO, urutan data pada queue menjadi terbalik.

Kesimpulan: Program ini menunjukkan kombinasi penggunaan queue dan stack untuk membalik urutan data.

2.6 IterasiQueue_2511533012

```

package pekan4_2511533012;

import java.util.Iterator;
import java.util.LinkedList;
import java.util.Queue;

public class IterasiQueue_2511533012 {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
    }
}

```

```

Queue<String> q_3012 = new LinkedList<>();

q_3012.add("Pratikum");
q_3012.add("Struktur");
q_3012.add("Data");
q_3012.add("Dan");
q_3012.add("Algoritma");
Iterator<String> iterator = q_3012.iterator();
while (iterator.hasNext()) {
    System.out.println(iterator.next() + " ");
}
}

```

Output :

```

ja <terminated> IterasiQ
Pratikum
Struktur
Data
Dan
Algoritma

```

Analisis :

Program ini menunjukkan cara melakukan **iterasi pada queue** menggunakan Iterator.

Beberapa data string dimasukkan ke dalam queue, kemudian ditampilkan satu per satu menggunakan perulangan while dan method hasNext(). Program ini memperlihatkan bagaimana seluruh isi queue dapat diakses secara berurutan.

Kesimpulan: Program ini menjelaskan proses iterasi data pada queue.

BAB III

KESIMPULAN

3.1 Ringkasan

Pada praktikum pekan ke-4, mahasiswa mempelajari implementasi struktur data queue menggunakan array dan LinkedList. Praktikum mencakup operasi dasar queue seperti enqueue, dequeue, front, rear, dan iterasi data. Selain itu, mahasiswa juga mempelajari penerapan queue bersama stack dalam proses membalik urutan data.

3.2 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa struktur data **Queue** sangat berguna dalam pengelolaan data yang bekerja secara berurutan dengan prinsip FIFO. Implementasi queue menggunakan array maupun LinkedList memiliki fungsi yang sama, namun berbeda dalam cara pengelolaan data dan memori. Praktikum ini membantu mahasiswa memahami konsep queue serta penerapannya dalam program Java.

DAFTAR PUSTAKA

- [1] Oracle, "The Java Tutorials," 2023. [Online]. Available: <https://docs.oracle.com/javase/tutorial/>.
- [2] H. M. Deitel dan P. J. Deitel, Java: How to Program, 10th ed. Boston: Pearson, 2015

Tugas Pratikum

1. Website_2511533012.java

```
package pekan4_2511533012;
import java.util.Scanner;

public class AntrianLoket_2511533012 {
    String queue[];
    int front;
    int rear;
    int max;

    public AntrianLoket_2511533012(int size) {
        max = size;
        queue = new String[max];
        front = -1;
        rear = -1;
    }

    boolean isEmpty_3012() {
        return rear == -1;
    }

    boolean isFull_3012() {
        return rear == max - 1;
    }

    void enqueue_3012(String data) {
        if (isFull_3012()) {
            System.out.println("Antrian penuh!");
        } else {
            if (isEmpty_3012()) {
                front = 0;
            }
            rear++;
            queue[rear] = data;
            System.out.println("Data berhasil ditambahkan ke antrian");
        }
    }

    void dequeue_3012() {
        if (isEmpty_3012()) {
            System.out.println("Antrian kosong!");
        } else {
            System.out.println(queue[front] + " telah dilayani");

            if (front == rear) {
                front = rear = -1;
            } else {
                for (int i = front; i < rear; i++) {
                    queue[i] = queue[i + 1];
                }
            }
        }
    }
}
```

```

        }
        rear--;
    }
}

void display_3012() {
    if (isEmpty_3012()) {
        System.out.println("Antrian kosong!");
    } else {
        System.out.println("Isi antrian:");
        for (int i = front; i <= rear; i++) {
            System.out.println((i + 1) + ". " + queue[i]);
        }
    }
}

void Reverse_3012() {
    if (isEmpty_3012()) {
        System.out.println("Antrian kosong!");
    } else {
        int awal = front;
        int akhir = rear;

        while (awal < akhir) {
            String temp = queue[awal];
            queue[awal] = queue[akhir];
            queue[akhir] = temp;

            awal++;
            akhir--;
        }

        System.out.println("Antrian berhasil dibalik");
    }
}

public static void main(String[] args) {
    Scanner input = new Scanner(System.in);

    AntrianLoket_2511533012 antrian =
        new AntrianLoket_2511533012(10);

    int pilih;

    do {
        System.out.println("\n=== PROGRAM ANTRIAN LOKET ===");
        System.out.println("1. Tambah Antrian");
        System.out.println("2. Hapus Antrian");
        System.out.println("3. Tampilkan Antrian");
        System.out.println("4. Reverse");
        System.out.println("5. Keluar");
        System.out.print("Pilih menu: ");
    }
}

```

```

        pilih = input.nextInt();
        input.nextLine();

        switch (pilih) {
            case 1:
                System.out.print("Masukkan nama pelanggan: ");
                String nama = input.nextLine();
                antrian.enqueue_3012(nama);
                break;

            case 2:
                antrian.dequeue_3012();
                break;

            case 3:
                antrian.display_3012();
                break;

            case 4:
                antrian.Reverse_3012();
                antrian.display_3012();
                break;

            case 5:
                System.out.println("Program selesai");
                break;

            default:
                System.out.println("Menu tidak tersedia!");
        }

        while (pilih != 5);

        input.close();
    }
}

```

2. Output dan Penjelasan Syntax

```
=== PROGRAM ANTRIAN LOKET PIKHUL ===  
1. Tambah Antrian  
2. Hapus Antrian  
3. Tampilkan Antrian  
4. Reverse  
5. Keluar  
Pilih menu:
```

```
5. Keluar  
Pilih menu: 1  
Masukkan nama pelanggan: Pikhul  
Data berhasil ditambahkan ke antrian
```

```
=== PROGRAM ANTRIAN LOKET PIKHUL ===  
1. Tambah Antrian  
2. Hapus Antrian  
3. Tampilkan Antrian  
4. Reverse  
5. Keluar  
Pilih menu: 1  
Masukkan nama pelanggan: Jonh  
Data berhasil ditambahkan ke antrian
```

```
=== PROGRAM ANTRIAN LOKET PIKHUL ===  
1. Tambah Antrian  
2. Hapus Antrian  
3. Tampilkan Antrian  
4. Reverse  
5. Keluar  
Pilih menu: 3  
Isi antrian:  
1. Pikhul  
2. Jonh
```

```
12  
ja  
=== PROGRAM ANTRIAN LOKET PIKHUL ===  
1. Tambah Antrian  
2. Hapus Antrian  
3. Tampilkan Antrian  
4. Reverse  
5. Keluar  
Pilih menu: 2  
Pikhul telah dilayani
```

```
va  
0  
12  
ja  
=== PROGRAM ANTRIAN LOKET PIKHUL ===  
1. Tambah Antrian  
2. Hapus Antrian  
3. Tampilkan Antrian  
4. Reverse  
5. Keluar  
Pilih menu: 4  
Antrian berhasil dibalik  
Isi antrian:  
1. rafi  
2. Jonh
```

```
=== PROGRAM ANTRIAN LOKET PIKHUL ===  
1. Tambah Antrian  
2. Hapus Antrian  
3. Tampilkan Antrian  
4. Reverse  
5. Keluar  
Pilih menu: 5  
Program selesai
```

Penjelasan:

Program ini dibuat untuk mengimplementasikan struktur data Queue (antrian) menggunakan array dalam bahasa Java. Program digunakan untuk

mensimulasikan antrian pelanggan pada loket pelayanan dengan konsep FIFO (First In First Out), yaitu data yang pertama masuk akan menjadi data pertama yang keluar.

Pada bagian awal program terdapat `import java.util.Scanner;` yang digunakan agar program dapat menerima input dari pengguna melalui keyboard. Setelah itu dibuat class `AntrianLoket_2511533012` sebagai class utama program sesuai aturan penamaan yang menggunakan NIM.

Di dalam class terdapat variabel `queue[]` yang berfungsi sebagai tempat penyimpanan data pelanggan dalam bentuk array. Variabel `front` digunakan untuk menunjukkan posisi antrian paling depan, sedangkan `rear` digunakan untuk menunjukkan posisi antrian paling belakang. Variabel `max` digunakan untuk menentukan kapasitas maksimal antrian.

Constructor `AntrianLoket_2511533012(int size)` digunakan untuk menginisialisasi ukuran array antrian serta memberikan nilai awal `front` dan `rear` yaitu `-1`. Nilai `-1` menandakan bahwa antrian masih kosong dan belum memiliki data.

Method `isEmpty()` digunakan untuk mengecek apakah antrian kosong. Method ini akan mengembalikan nilai `true` jika `rear` bernilai `-1`. Sedangkan method `isFull()` digunakan untuk mengecek apakah antrian penuh dengan cara memeriksa apakah nilai `rear` sudah mencapai indeks terakhir array.

Method `enqueue()` digunakan untuk menambahkan data pelanggan ke dalam antrian. Jika antrian penuh maka program akan menampilkan pesan bahwa antrian penuh. Jika masih kosong maka `front` diubah menjadi `0`, kemudian nilai `rear` ditambah satu dan data pelanggan disimpan ke dalam array `queue`.

Method `dequeue()` digunakan untuk menghapus data pelanggan paling depan sesuai konsep FIFO. Jika antrian kosong maka program menampilkan pesan "Antrian kosong". Jika terdapat data maka pelanggan paling depan akan dihapus dan ditampilkan sebagai pelanggan yang telah dilayani. Setelah itu data dalam array digeser ke depan agar posisi antrian tetap rapi.

Method `display()` digunakan untuk menampilkan seluruh isi antrian. Program akan melakukan perulangan dari indeks `front` hingga `rear` untuk menampilkan semua nama pelanggan yang ada dalam antrian.

Method `Reverse()` digunakan untuk membalik urutan isi antrian. Proses dilakukan dengan menukar data dari depan dan belakang menggunakan variabel sementara hingga seluruh isi antrian berhasil dibalik.

Pada method `main`, program menampilkan menu utama yang terdiri dari tambah antrian, hapus antrian, tampilkan antrian, reverse, dan keluar program. Pengguna dapat memilih menu sesuai kebutuhan dan program akan menjalankan method yang sesuai dengan pilihan tersebut. Program akan terus berjalan menggunakan perulangan `do while` sampai pengguna memilih menu keluar.